

Final Presentation

**ProblemC: Incremental Placement Optimization Beyond Detailed Placement:
Simultaneous Gate Sizing, Buffering, and Cell Relocation**

Presenter: 巫品佑

Date: June 20, 2025

Outline

1

Related Works

2

Algorithm

3

Implementation Method

4

Result

Outline

1

Related Works

2

Algorithm

3

Implementation Method

4

Experiment Result

RLPlace DATE'22, by Nvidia

- ❑ Deep RL for detailed placement
- ❑ Objective: Minimize HPWL
- ❑ Method:
 - ❑ Stage 1: coarse-grained placement
 - ❑ Using RL to switch grids within each window
 - ❑ Stage 2: fine-grained refinement
 - ❑ Stage 1 may cause cell overlap
 - ❑ Solved by Satisfiability Modulo Theories (SMT) solver
- ❑ Notice that objective includes displacement

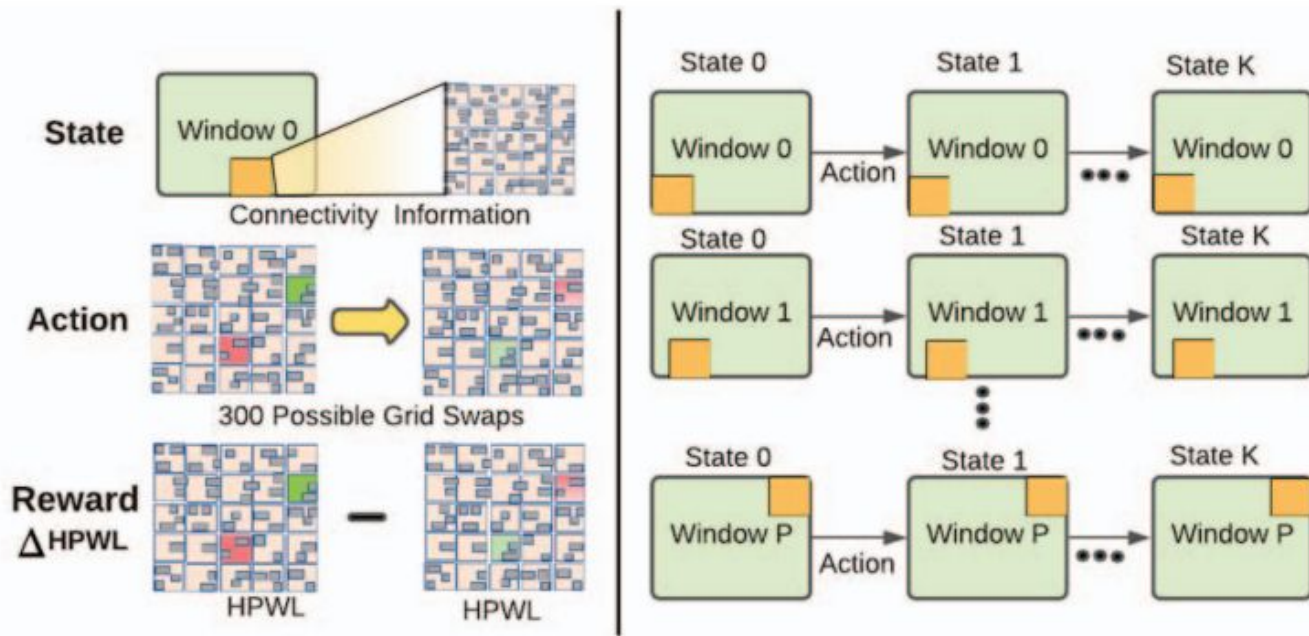


Fig. 1. RLPlace. Reward:HPWL, Penalty: displacement

Differentiable Gate Sizing

ICCAD'24, by Peking University

Linear interpolation of gates

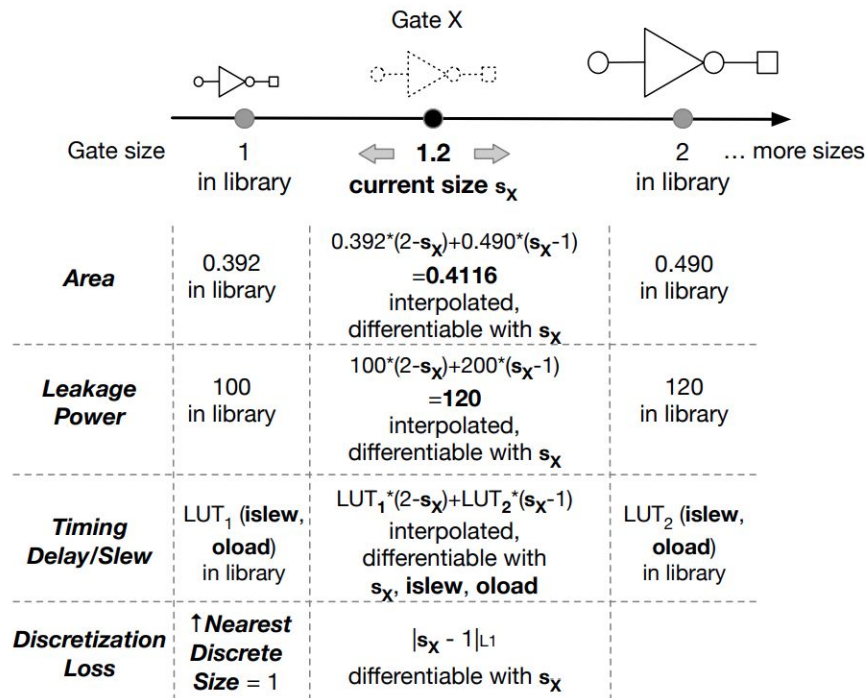


Fig. 2. Differentiable Gate Sizing

Outline

1

Related Works

2

Algorithm

3

Implementation Method

4

Experiment Result

Timing - Greedy Algorithm

- ❑ Goal: timing repair along a given **path with negative slack** of the given initial placement
- ❑ Calculate the load delay of each gate along the path, then sort in **descending order**
- ❑ Determine how many repair actions should be attempted in this pass based on how severe the timing violation is.
 - ❑ Repair action: **gate resizing, buffer insertion, or cell relocation**
- ❑ For each prioritized gate, it iteratively tries a sequence of repair actions, stopping at the first successful change
- ❑ The process is limited by the allowed # of repair actions to **balance between insufficient fixing and over-correction**

Window-based incremental optimization Framework

- ❑ Including the methods provided in the Related Works, aiming to do some integration and extension to apply to this contest
- ❑ Goal: iteratively refines placement and sizing decisions to improve design quality under timing and congestion constraints
- ❑ Parameters
 - ❑ window size **w** (with initial value)
 - ❑ maximum window size **w_max**
 - ❑ window incremental value **w_inc**
 - ❑ current # of non-improving iterations **counter**
 - ❑ maximum # of non-improving iterations **rot**

Window-based incremental optimization Framework

- ❑ Per-window loop:
 - ❑ RL-based cell relocation
 - ❑ analytical gate sizing
 - ❑ static timing analysis (STA) using the INSTA engine, which also updates the congestion parameters.
- ❑ Cost Check
 - ❑ New cost < best: save as best, reset **counter**
 - ❑ otherwise: **counter++**

Window-based incremental optimization Framework

- ❑ Stopping for current Per-window loop
 - ❑ End when **counter** reaches **rot** (no further gain)
- ❑ Window Growth
 - ❑ Increase window size **w** by **w_inc** and enter the next Per-window loop
 - ❑ Repeat until window size **w** reaches **w_max**
- ❑ Final Output
 - ❑ changelist corresponding to the lowest cost configuration

Outline

1

Related Works

2

Algorithm

3

Implementation Method

4

Experiment Result

Optimization Flow

- ❑ Tool: OpenROAD
 - placement and routing analysis, advanced timing optimization and power estimation
- ❑ Placement-level Parasitic Estimation
- ❑ Iterations:
 - ❑ Select **gate resizing, buffer insertion, cell relocation, or combinations**
 - ❑ Run **repair timing** with the **-setup** flag
 - Goal: fix timing violations related to setup negative slack (by Greedy Algorithm)
 - ❑ Ensure legal and optimized placement after each iteration(ex. no cell overlap)
 - ❑ Record **Total Negative Slack (TNS), Worst Negative Slack (WNS), power, runtime and wirelength(HPWL)**

How to Implement

- ❑ Install Docker and install ubuntu-22.04 by the following instruction:

```
$ docker run -it ubuntu:22.04
```

- ❑ Create a container named openroad-env.

- ❑ Copy local project files into the container using instruction **docker cp**

- ❑ Execute the .tcl script inside the container by the following instruction:

```
root@#: openroad /workspace/openRoad_eval_script/eval_def.tcl
```

- ❑ Install VcXsrv (X server) on Windows and launch it

- ❑ Open OpenROAD GUI by the following instruction:

```
root@#: openroad -gui
```

Outline

1

Related Works

2

Algorithm

3

Implementation Method

4

Experiment Result

Experiment Result

- Execute the .tcl script inside the container by the following instruction:

```
root@#: /workspace/openRoad_eval_script# openroad eval_def.tcl
```

```
tns -100709.30
wns -339.64
```

Group	Internal Power	Switching Power	Leakage Power	Total Power (Watts)	
Sequential	8.24e-03	2.73e-03	8.83e-06	1.10e-02	19.8%
Combinational	1.85e-02	2.58e-02	1.91e-05	4.43e-02	80.2%
Clock	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Macro	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Pad	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Total	2.67e-02 48.4%	2.85e-02 51.6%	2.79e-05 0.1%	5.53e-02	100.0%

Fig. 4. Baseline Result

Experiment Result

- Execute the .tcl script inside the container by the following instruction:

```
root@#: /workspace/openRoad_eval_script# openroad eval_def.tcl
```

Method	TNS	TNS Impr. (%)	WNS	Power (W)	HPWL	Runtime (s)
OpenROAD [3] v2.0-17598-ga008522d8						
Baseline	-100709.3	0.00	-339.64	5.53E-02	32394.3u	8.463
Repair (Not Legal)	-11687.35	-88.39	-45.12	5.80E-02	–	93.272
Buffer + Resize (Not Legal)	-12185.75	-87.90	-46.33	5.77E-02	–	71.9
Buffer + Resize + DP	-15592.41	-84.52	-81.69	5.91E-02	40027.2u	71.237
Only Resize (Not Legal)	-13444.71	-86.65	-57.79	5.71E-02	–	124.416
Only Resize + DP	-15946.53	-84.17	-72.10	5.77E-02	35290.9u	121.441
DP + Buffer + Resize + DP	-14786.6	-85.32	-85.95	5.92E-02	44278.3u	79.852
OpenROAD [3] v2.0-22086-g1f25bd6856						
DP + Buffer + Resize + DP + Improved placement	-12829.12	-87.26	-71.20	5.75E-02	34396.8u	62.826

Fig. 5. Experiment Result

Experiment Result

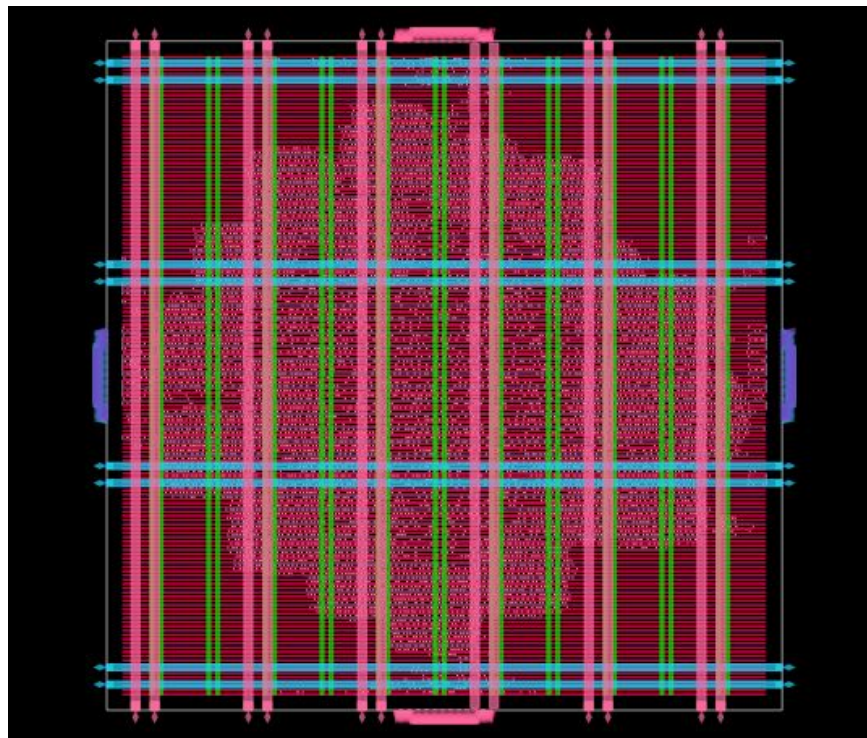


Fig. 6. Layout of Baseline

Experiment Result

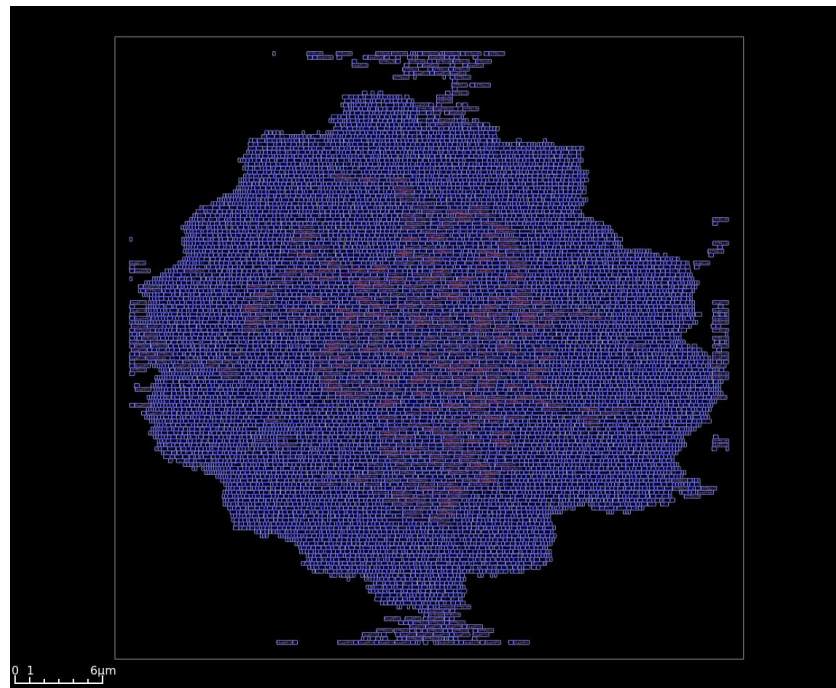


Fig. 7. Layout of repair timing and legal, disable nets(including initial detailed placement)

Future Work

